

DepoMetrik Faz 3: Çevrimdışı Öncelikli FinTek Altyapısı, Kimlik Doğrulama ve Açık Bankacılık Entegrasyon Mimari Raporu

Mimari Giriş ve Hesaplama Altyapısı

DepoMetrik platformu, 04.06.2026 tarihi itibarıyla ürün geliştirme yol haritasında ilk iki fazı başarıyla tamamlamış ve çevrimdışı öncelikli (offline-first) mimarinin en kritik entegrasyon aşaması olan Faz 3 adımına geçiş yapmıştır. İlk iki faz kapsamında yerel veritabanı şemaları, depodan depoya tüketim algoritmaları, cihaz üzeri OCR (yakıt fişi tarama) ve kredi kartı PDF/SMS maskeleye motorları başarıyla devreye alınmıştır. Faz 3 ise uygulamanın dış finansal sistemler, resmi devlet servisleri ve bulut senkronizasyon katmanıyla entegre olduğu dönemi temsil etmektedir.

Sistemin çekirdeğinde yer alan tüketim hesaplama motoru, iki temel matematiksel formüle dayanmaktadır. Depodan depoya (Full-to-Full) tüketim algoritması, iki ardışık tam dolum ($is_full_tank = true$) işlemi arasındaki yakıt tüketimini litre bazında şu formülle hesaplar:

$$C_i = \frac{L_{litre}}{K_i - K_{i-1}} \times 100$$

Kısmi yakıt alımlarının gerçekleştiği ve ardışık tam dolumların bulunmadığı izleme periyotlarında ise kayan ağırlıklı ortalama (Rolling Average) formülü devreye girmektedir:

$$C_{rolling} = \frac{\sum_{j=1}^n L_j}{K_n - K_0} \times 100$$

Her iki tüketim algoritmasının da kararlı ve doğru çalışabilmesi için kilometre sayacının kronolojik olarak kesin bir şekilde artan ($K_i > K_{i-1}$) yapıda olması teknik bir zorunluluktur. Bu kuralın ihlal edildiği durumlarda sistem, veri bütünlüğünü korumak adına derhal bir `InvalidOdometerException` fırlatacak şekilde yapılandırılmıştır.

Faz 3 kapsamında entegrasyonu gerçekleştirilecek olan veri tabanı ilişkisel şeması, PostgreSQL (Supabase) ve Drift (SQLite) arasında çift yönlü senkronizasyon sağlayan PowerSync motoru ile yönetilmektedir. Sistemde tanımlı yedi temel tablonun ilişkisel kısıtlamaları ve veri tipleri şu şekildedir:

- **profiles:** user_id (UUID - PK), email (text, nullable), created_at (timestamp), premium_status (bool, default false).
- **vehicles:** vehicle_id (UUID - PK), user_id (FK -> profiles), plate (varchar, standart Türkiye plakası regex doğrulamalı), brand (varchar), model (varchar), fuel_type (enum: BENZIN, DİZEL, LPG, ELEKTRİK), initial_odometer (int >= 0), current_odometer (int >= 0).
- **stations:** station_id (UUID - PK), brand_name (varchar), latitude (double), longitude (double), geom (PostGIS Point, 4326), city (varchar), district (varchar).
- **refuelings:** refueling_id (UUID - PK), vehicle_id (FK -> vehicles, cascade), station_id (FK -> stations, set null), liters (double > 0), unit_price (double > 0), total_price (double > 0), odometer (int >= 0), purchase_date (timestamp), is_full_tank (bool, default true).
- **card_transactions:** transaction_id (UUID - PK), user_id (FK -> profiles), refueling_id (FK -> refuelings, set null), transaction_date (date), amount (double > 0), merchant_name (varchar), source (enum: PDF, SMS, API).

- **campaigns:** campaign_id (UUID - PK), user_id (FK -> profiles), bank_name (varchar), station_brand (varchar), target_tx_count (int > 0), current_tx_count (int default 0), reward_amount (double > 0), expiry_date (timestamp).
- **obd_readings:** reading_id (UUID - PK), vehicle_id (FK -> vehicles, cascade), odometer_value (int >= 0), fuel_level_ratio (double, 0.0 - 1.0), recorded_at (timestamp).

Açık Bankacılık (BKM GEÇİT) üzerinden akan veri setleri, doğrudan card_transactions tablosuna source = 'API' değeriyle yazılmakta ve yerel OCR ile taranan refuelings tablosundaki fiş kayıtlarıyla ilişkisel olarak eşleştirilmektedir.

NVİ SOAP Entegrasyonu: Supabase Edge Function Tasarımı

Nüfus ve Vatandaşlık İşleri Genel Müdürlüğü (NVİ) tarafından sunulan Kimlik Paylaşımı Sistemi Kamu Servisi (KPSPublic), TCKN doğrulaması için resmi SOAP API protokolünü kullanmaktadır. Açık Bankacılık (QPAY) servis çağrılarında önce suiistimalleri önlemek, işlem maliyetlerini düşürmek ve MASAK uyumluluğunu yasal düzeyde garanti altına almak amacıyla kullanıcıların kimlik bilgileri bu servis üzerinden doğrulanmaktadır.

SOAP Protokolü ve İstek Yapısı

KPSPublic servisi, <https://tckimlik.nvi.gov.tr/Service/KPSPublic.asmx> adresinde konumlanmıştır. İstekler POST metodunu kullanmalı ve HTTP başlıklarında Content-Type: text/xml; charset=utf-8 ile SOAPAction: "http://tckimlik.nvi.gov.tr/WS/TCKimlikNoDogrula" alanlarını barındırmalıdır. SOAP istek gövdesi (envelope), parametre isimlerine ve namespaces alanlarına karşı son derece hassastır.

Ayrıca, NVİ veritabanında isim ve soyisim eşleştirmelerinin başarılı olabilmesi için gönderilen metinlerin Türkçe karakter kurallarına uygun olarak tamamen büyük harfe dönüştürülmesi gerekmektedir. JavaScript dilinin standart .toUpperCase() metodu, Türkçe karakterlerde (örneğin "İ" karakterinin "I" yerine "i" yapılması gibi) hatalı dönüşümlere yol açtığı için özel bir dönüşüm fonksiyonunun kullanılması teknik bir zorunluluktur.

Supabase Edge Function (Deno/TypeScript) Gerçekleştirimi

Aşağıda sunulan Deno tabanlı Edge Function kodu, @std/xml kütüphanesini kullanarak SOAP servisinden dönen XML yanıtını parse eder ve Flutter istemcisine standart JSON formatında doğrulanmış boolean bir yanıt döner.

```
import { serve } from "https://deno.land/std@0.224.0/http/server.ts";
import { parse } from "https://deno.land/std@0.224.0/xml/mod.ts";
```

```
const NVI_SOAP_ENDPOINT =
  "https://tckimlik.nvi.gov.tr/Service/KPSPublic.asmx";
```

```
interface VerificationPayload {
  tcKimlikNo: number;
  ad: string;
  soyad: string;
  dogumYili: number;
```

```

}

// Türkçe karakter duyarlı büyük harfe dönüştürme fonksiyonu
function convertToTurkishUpperCase(text: string): string {
  const map: { [key: string]: string } = {
    'ç': 'Ç', 'ğ': 'Ğ', 'ı': 'İ', 'i': 'İ', 'ö': 'Ö', 'ş': 'Ş', 'ü':
    'Ü',
  };
  return text
    .replace(/[çğiiöşü]/g, (match) => map[match])
    .toUpperCase()
    .trim();
}

serve(async (req) => {
  // CORS Politika Yapılandırması
  if (req.method === "OPTIONS") {
    return new Response("ok", {
      headers: {
        "Access-Control-Allow-Origin": "*",
        "Access-Control-Allow-Methods": "POST",
        "Access-Control-Allow-Headers": "authorization, x-client-info,
apikey, content-type",
      },
    });
  }

  try {
    const payload: VerificationPayload = await req.json();
    const { tcKimlikNo, ad, soyad, dogumYili } = payload;

    if (!tcKimlikNo || !ad || !soyad || !dogumYili) {
      return new Response(
        JSON.stringify({ error: "Eksik parametre gönderildi. TCKN, Ad,
Soyad ve Doğum Yılı zorunludur." }),
        { status: 400, headers: { "Content-Type": "application/json" } }
      );
    }

    const adUpper = convertToTurkishUpperCase(ad);
    const soyadUpper = convertToTurkishUpperCase(soyad);

    // Deterministik XML Şeması İnşası
    const soapEnvelope = `<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">

```

```

<soap:Body>
  <TCKimlikNoDogrula xmlns="http://tckimlik.nvi.gov.tr/WS">
    <TCKimlikNo>${tcKimlikNo}</TCKimlikNo>
    <Ad>${adUpper}</Ad>
    <Soyad>${soyadUpper}</Soyad>
    <DogumYili>${dogumYili}</DogumYili>
  </TCKimlikNoDogrula>
</soap:Body>
</soap:Envelope>`;

// NVİ SOAP Servisine İstek Gönderilmesi
const response = await fetch(NVI_SOAP_ENDPOINT, {
  method: "POST",
  headers: {
    "Content-Type": "text/xml; charset=utf-8",
    "SOAPAction":
"http://tckimlik.nvi.gov.tr/WS/TCKimlikNoDogrula",
  },
  body: soapEnvelope,
});

if
(!respons[span_16] (start_span) [span_16] (end_span) [span_18] (start_span)
[span_18] (end_span)e.ok) {
  throw new Error(`NVİ SOAP Servis Hatası. Durum Kodu:
${response.status}`);
}

const xmlResponseText = await response.text();

// XML Yanıtının DOM Yapısına Dönüştürülmesi
const parsedDocument = parse(xmlResponseText) as any;
[span_24] (start_span) [span_24] (end_span)

// Güvenli XML Hiyerarşisi Gezinimi
const envelope = parsedDocument["soap:Envelope"] ||
parsedDocument["Envelope"];
const body = envelope? (envelope || envelope) : null;
const responseNode = body? (body || body) : null;
const resultNode = responseNode? (responseNode || responseNode) :
null;

const validationResult = resultNode === "true" ||
resultNode?.["#text"] === "true";
[span_25] (start_span) [span_25] (end_span) [span_26] (start_span) [span_26]
(end_span)

return new Response(

```

```

        JSON.stringify({ success: validationResult })),
        {
            status: 200,
            headers: {
                "Content-Type": "application/json",
                "Access-Control-Allow-Origin": "*",
            },
        },
    );
} catch (error: any) {
    return new Response(
        JSON.stringify({ error: error.message || "Bilinmeyen bir hata
oluştı." })),
        {
            status: 500,
            headers: {
                "Content-Type": "application/json",
                "Access-Control-Allow-Origin": "*",
            },
        },
    );
}
});

```

Çevrimdışı Öncelikli Mimaride "Çevrimiçi" Servis Yönetimi

PowerSync entegrasyonu sayesinde DepoMetrik uygulaması, veri tabanına yazma işlemlerini ağ bağlantısından bağımsız olarak yerel SQLite veritabanına yapabilmekte ve bağlantı geri geldiğinde arka planda çift yönlü senkronizasyonu tamamlamaktadır. Bununla birlikte, NVİ doğrulaması ve BKM GEÇİT rıza onaylama adımları, doğası gereği anlık ve reaktif HTTP protokolleri üzerinden yürütülen, ertelenemez ve çevrimdışı önbelleğe alınamaz süreçlerdir. Bu durum, kullanıcı deneyiminde (UX) akıcı ve kesintisiz bir geçiş katmanı tasarlanmasını gerektirir.

Reaktif Bağlantı İzleme ve Durum Yönetimi

Uygulamanın anlık bağlantı durumu, işletim sisteminin ağ arabirimini dinleyen dinamik bir Stream ile takip edilir. Eğer kullanıcı çevrimdışı konumdaysa, sadece "çevrimiçi zorunlu" olan bu adımlar arayüz düzeyinde pasifleştirilir, kullanıcı bilgilendirici görsel uyarılarla yönlendirilir ve gereksiz ağ isteklerinin önüne geçilir. Bu süreç Flutter tarafında Clean Architecture ilkelerine uygun olarak BLoC mimarisiyle yönetilir.

Aşağıda, ağ bağlantı durumunu takip eden ve çevrimiçi zorunlu servis akışlarını kesintiye uğratmadan yöneten reaktif IdentityVerificationBloc gerçekleştirimi yer almaktadır:

```

import 'dart:async';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:connectivity_plus/connectivity_plus.dart';

```

```

import 'package:http/http.dart' as http;
import 'dart:convert';

// --- BLoC Events ---
abstract class IdentityEvent {}

class ConnectivityChangedEvent extends IdentityEvent {
  final List<ConnectivityResult> results;
  ConnectivityChangedEvent(this.results);
}

class ExecuteNviVerificationEvent extends IdentityEvent {
  final String tckn;
  final String name;
  final String surname;
  final int birthYear;

  ExecuteNviVerificationEvent({
    required this.tckn,
    required this.name,
    required this.surname,
    required this.birthYear,
  });
}

// --- BLoC States ---
abstract class IdentityState {}

class IdentityUnitialized extends IdentityState {}

class SystemConnectivityStatus extends IdentityState {
  final bool isOnline;
  SystemConnectivityStatus({required this.isOnline});
}

class VerificationInProgress extends IdentityState {}

class VerificationCompleted extends IdentityState {
  final bool isVerified;
  VerificationCompleted({required this.isVerified});
}

class VerificationFailed extends IdentityState {
  final String error;
  VerificationFailed({required this.error});
}

// --- BLoC Controller ---

```

```

class IdentityVerificationBloc extends Bloc<IdentityEvent,
IdentityState> {
    final Connectivity _connectivity = Connectivity();
    StreamSubscription<List<ConnectivityResult>>?
    _connectivitySubscription;
    bool _isCurrentlyOnline = false;

    IdentityVerificationBloc() : super(IdentityUnitialized()) {
        on<ConnectivityChangedEvent>((event, emit) {
            _isCurrentlyOnline = event.results.any((res) => res!=
ConnectivityResult.none);
            emit(SystemConnectivityStatus(isOnline: _isCurrentlyOnline));
        });

        on<ExecuteNviVerificationEvent>(_onExecuteNviVerification);

        _connectivitySubscription =
    _connectivity.onConnectivityChanged.listen((results) {
        add(ConnectivityChangedEvent(results));
    });
    }

    Future<void> _onExecuteNviVerification(
        ExecuteNviVerificationEvent event,
        Emitter<IdentityState> emit,
    ) async {
        if (!_isCurrentlyOnline) {
            emit(VerificationFailed(
                error: "İnternet bağlantısı bulunamadı. Kimlik doğrulaması
için aktif ağ bağlantısı gereklidir.",
            ));
            return;
        }

        emit(VerificationInprogress());

        try {
            final response = await http.post(
Uri.parse("https://api.depometrik.com/functions/v1/nvi-verification"),
                headers: {
                    "Content-Type": "application/json",
                    "Authorization": "Bearer SERVICE_ROLE_RESTRICTED_TOKEN",
                },
                body: jsonEncode({
                    "tcKimlikNo": int.parse(event.tckn),
                    "ad": event.name,
                    "soyad": event.surname,

```

```

        "dogumYili": event.birthYear,
    })),
);

if (response.statusCode == 200) {
    final Map<String, dynamic> responseData =
jsonDecode(response.body);
    final bool isSuccess = responseData["success"]?? false;
    emit(VerificationCompleted(isVerified: isSuccess));
} else {
    emit(VerificationFailed(
        error: "Doğrulama sunucusu yanıt vermedi. Durum Kodu:
${response.statusCode}",
    ));
}
} catch (exception) {
    emit(VerificationFailed(error: "Ağ geçidi bağlantı hatası:
${exception.toString()}"));
}
}

@override
Future<void> close() {
    _connectivitySubscription?.cancel();
    return super.close();
}
}

```

KVKK ve MASAK Uyumlu Veri Şifreleme ve Güvenlik Standartları

Kişisel Verilerin Korunması Kanunu (KVKK) Madde 4 kapsamında "Veri Minimizasyonu" ve "Amaçla Sınırlılık" ilkeleri gereği, kullanıcıların TCKN bilgileri ve hassas finansal detayları merkezi sunucularda açık metin olarak asla tutulmamalı, sızıntı ihtimalleri mimari düzeyde sıfıra indirilmelidir. Bu bağlamda, platformun yerel ve bulut veri depolama katmanları endüstri standardı kriptografik algoritmalarla uçtan uca koruma altına alınmıştır.

Veritabanı Şifreleme Teknolojilerinin Mimari Karşılaştırması

Şifreleme stratejilerinin belirlenmesinde, Supabase ve PostgreSQL ekosisteminde yer alan alternatif çözümler derinlemesine analiz edilmiştir:

Mimari Katman	Güvenlik Düzeyi	Anahtar Yönetim Mekanizması	KVKK / MASAK Uyumluluğu	Performans ve Kullanılabilirlik Etkisi
pgsodium	Orta (Bağımlılık)	Veritabanı ile aynı	Düşük (Sızma	Yüksek

Mimari Katman	Güvenlik Düzeyi	Anahtar Yönetim Mekanizması	KVKK / MASAK Uyumluluğu	Performans ve Kullanılabilirlik Etkisi
(Transparent Column Encryption - TCE)	riski yüksek).	diskte veya harici KMS'de tek anahtar saklama.	anında tek anahtar sızıntısı tüm kayıtları açar).	performans kaybı, versiyon yükseltme ve taşıma zorlukları.
Supabase Vault	Yüksek (Statik sırlar için optimize).	Postgres şemasında asimetrik anahtar korumalı vault.secrets.	Yüksek (Yalnızca API anahtarı ve statik entegrasyon token'ları için).	Düşük hacimli sır depolama için idealdir, yüksek hacimli kullanıcı PII verisine uygun değildir.
pgcrypto (PostgreSQL Extensions)	Çok Yüksek (Tablo ve satır bazlı esnek şifreleme).	Uygulama katmanından gönderilen parola veya çevre değişkeni tabanlı anahtar.	En Yüksek (Bulut yöneticilerinin bile veriyi doğrudan okuması engellenir).	SQL sorgularında her filtrelemede şifre çözme maliyeti oluşturur.
Drift + SQLite3MultipleCiphers (Yerel DB)	En Yüksek (Cihaz fiziksel güvenliği).	Mobil işletim sistemi donanım katmanı (iOS Keychain, Android Keystore).	En Yüksek (Sıfır Bilgi - Zero Knowledge prensibiyle buluta gitmeden şifreleme).	Yerel donanım hızlandırma sayesinde performans etkisi ihmal edilebilir düzeydedir.

Supabase tarafından pgsodium eklentisinin aşamalı olarak kaldırılacağı (pending deprecation) resmi olarak duyurulmuştur. Sütun seviyesinde şifrelemenin artık tamamen SQL seviyesine çekilmesi ve pgsodium yerine pgcrypto ile yerel cihaz şifrelemesi hibrit yapısının kurulması önerilmektedir. Supabase Vault ise yüksek hacimli kullanıcı kimlik verileri (PII) yerine, sadece harici QPAY API kimlik doğrulama anahtarları gibi düşük hacimli sistem sırlarını saklamak için kullanılmalıdır.

Yerel Veritabanının SQLite3MultipleCiphers ile Kriptolanması

Drift veritabanı altyapısında SQLCipher yerine modern, kararlı ve çoklu şifreleme protokollerini destekleyen SQLite3MultipleCiphers (sqlite3mc) mimarisine geçiş yapılmıştır.

Projenin kök dizinindeki pub[span_48](start_span)[span_48](end_span)spec.yaml dosyası üzerinde, derleme kancaları (build hooks) aracılığıyla SQLite kütüphanesinin standart sürüm yerine sqlite3mc kaynak kodlarıyla derlenmesi şu konfigürasyon ile sağlanır:

```
dependencies:
  drift: ^2.33.0
  sqlite3: ^3.0.0

hooks:
  user_defines:
    sqlite3:
      source: sqlite3mc
```

Yerel veritabanının ilk açılışında, cihazın donanımsal şifreleme modüllerinden elde edilen benzersiz anahtar, Drift motoruna geçilmeden önce PRAGMA key komutu vasıtasıyla veritabanı motoruna enjekte edilmektedir.

Aşağıdaki Dart kodu, hem şifreli bir veritabanı oluşturma sürecini hem de daha önceden şifrelenmemiş düz metin (plaintext) formatındaki bir veritabanını, veri kaybına yol açmadan VACUUM INTO ve PRAGMA rekey mekanizmalarıyla şifreli yapıya dönüştürme mantığını içermektedir:

```
import 'dart:io';
import 'package:drift/drift.dart';
import 'package:drift/native.dart';
import 'package:path_provider/path_provider.dart';
import 'package:path/path.dart' as p;
import 'package:sq_lite3/sq_lite3.dart';

bool _debugCheckHasCipher(Database database) {
  return database.select('PRAGMA cipher;').isNotEmpty;
}

String _escapeString(String
so[span_51] (start_span) [span_51] (end_span)urce) {
  return source.replaceAll('\'', '\\\'');
}

LazyDatabase
createEncryptedCo[span_52] (start_span) [span_52] (end_span)nnection(Stri
ng passphrase) {
  return LazyDatabase(() async {
    final documentsDirectory = await
getApplicationDocumentsDirectory();
    final databaseFile = File(p.join(documentsDirectory.path,
'depometrik_secure.db'));
    final plaintextFile = File(p.join(documentsDirectory.path,
'depometrik_plain.db'));

    // Eğer eski sürümden kalan şifrelenmemiş bir veritabanı varsa,
güvenli geçiş başlatılır
    if (await plaintextFile.exists() &&!await databaseFile.exists()) {
      final tempEncryptedFile = File('${databaseFile.path}.tmp');
      if (await tempEncryptedFile.exists()) {
        await tempEncryptedFile.delete();
      }

      // 1. Şifrelenmemiş veritabanı açılır ve temiz bir kopyası
çıkartılır
      final plainDb = sq_lite3.open(plaintextFile.path);
      plainDb.execute("VACUUM INTO
'$ {_escapeString(tempEncryptedFile.path)} '");
```

```

        plainDb.close();

        / [span_53] (start_span) [span_53] (end_span) / 2. Çıkartılan temiz
        kopya açılarak şifreleme anahtarı atanır
        final targetDb = sqlite3.open(tempEncryptedFile.path);
        targetDb.execute("PRAGMA rekey =
        '${_escapeString(passPhrase)}';");
        targetDb.close();

        [span_54] (start_span) [span_54] (end_span) // 3. Dosya işlemleri
        tamamlanarak eski düz metin veritabanı sistemden silinir
        await tempEncryptedFile.rename(databaseFile.path);
        await plaintextFile.delete();
    }

    return NativeDatabase.createInBackground(
        databaseFile,
        setup: (rawDb) {
            // SQLite3MultipleCiphers paketinin varlığı çalışma zamanında
            doğrulanır
            assert(_debugCheckHasCipher(rawDb));

            // Veritabanı [span_55] (start_span) [span_55] (end_span) oturumu
            şifreleme anahtarıyla yetkilendirilir
            rawDb.execute("PRAGMA key = '${_escapeString(passPhrase)}';");
        },
    );
    });
}

---
```

BKM GEÇİT WebView ve Güvenli Derin Bağlantı (Deep Link) Protokolü

Açık Bankacılık Ödeme Hizmetleri Veri Paylaşım Servisleri (ÖHVPS 2.0 / BKM GEÇİT) mimarisi kapsamında, kullanıcının hesap hareketlerine (özellikle kredi kartı akaryakıt işlemlerine) erişebilmek için lisanslı açık bankacılık entegratörü (QPAY) üzerinden bir rıza numarası (Consent ID) üretilir. Kullanıcı, bankasının Güçlü Kimlik Doğrulama (GKD) adımlarını tamamlaması amacıyla bankanın OAuth 2.0 giriş arayüzüne yönlendirilir.

Güvenli Callback ve App Link / Universal Link Yapılandırması

Kullanıcı bankacılık arayüzünde doğrulamasını başarıyla tamamladığında, banka sunucusu istemciyi önceden tanımlanmış olan

[https://a\[span_9\]\(start_span\)\[span_9\]\(end_span\)pi.depometrik.com/openbanking/callback](https://a[span_9](start_span)[span_9](end_span)pi.depometrik.com/openbanking/callback) adresine yönlendirir. Bu kritik adımda, kötü niyetli uygulamaların araya girerek rıza verilerini

çalmasını (App Hijacking) önlemek amacıyla, doğrulanmamış özel URI şemaları (Custom Scheme) yerine işletim sistemi düzeyinde dijital imza doğrulaması barındıran **Android App Links** ve **iOS Universal Links** protokolleri kullanılmalıdır.

Android Güvenlik Yapılandırması (AndroidManifest.xml)

İşletim sisteminin alan adı sahipliğini doğrulaması için android:autoVerify="true" parametresi eklenmiştir:

```
<manifest xmlns:android="http://schemas.xmlsoap.org/apk/res/android">
    <application>
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:launchMode="singleTask">

            <intent-filter android:autoVerify="true">
                <action android:name="android.intent.action.VIEW" />
                <category
                    android:name="android.intent.category.DEFAULT" />
                    <category
                        android:name="android.intent.category.BROWSABLE" />

                        <data android:scheme="https" />
                        <data android:host="api.depometrik.com" />
                        <data android:pathPrefix="/openbanking/callback" />
                    </intent-filter>
                </activity>
            </application>
        </manifest>
```

iOS Güvenli İlişkilendirme Yapılandırması (Runner.entitlements)

iOS cihazlarında ilgili alan adından gelen çağrılar yalnızca doğrulanmış DepoMetrik uygulamasına yönlendirilmesi sağlanır:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
"http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>com.apple.developer.associated-domains</key>
    <array>
        <string>applinks:api.depometrik.com</string>
    </array>
</dict>
</plist>
```

Uygulamanın web sunucusunda <https://api.depometrik.com/.well-known/assetlinks.json>

(Android) ve <https://api.depometrik.com/.well-known/apple-app-site-association> (iOS) dosyaları barındırılmalı ve bu dosyalar uygulamanın resmi mağaza kimlik bilgileriyle imzalanmalıdır.

CSRF Engelleme ve Durum Doğrulama Akışı

Güvenlik zaafiyetlerini ve sahte rıza onaylama girişimlerini (Cross-Site Request Forgery - CSRF) engellemek amacıyla, rıza süreci başlatıldığında istemci tarafından benzersiz bir kriptografik state belirteci üretilir ve bu belirteç yerel Drift veritabanında saklanır. Geri dönüş bağlantısı alındığında, URL içerisindeki state değeri ile yerelde saklanan değer karşılaştırılır. Eşleşme sağlanırsa, doğrulama işlemi sunucu katmanında tamamlanır.

Aşağıdaki Dart kodu, derin bağlantıyı (deep link) yakalayan, state parametresini doğrulayan ve başarılı açık bankacılık verilerini yerel card_transactions tablosuna entegre eden yapıyı göstermektedir:

```
import 'package:app_links/app_links.dart';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';

class OpenBankingLinkController {
  final AppLinks _appLinksParser = AppLinks();

  void listenToReturnCallbacks(BuildContext context, String
localSavedStateToken) {
    _appLinksParser.uriLinkStream.listen((Uri incomingUri) {
      if (incomingUri.host == 'api.depometrik.com' &&
          incomingUri.path.startsWith('/openbanking/callback')) {
        _processIncomingCallback(context, incomingUri,
localSavedStateToken);
      }
    }, onError: (Object networkError) {
      // Hata izleme logları burada yapılandırılacaktır
    });
  }

  Future<void> _processIncomingCallback(
    BuildContext context,
    Uri callbackUri,
    String originalStateToken,
  ) async {
    final String? receivedConsentId =
callbackUri.queryParameters['consent_id'];
    final String? receivedStateToken =
callbackUri.queryParameters['state'];
    final String? transactionStatus =
callbackUri.queryParameters['status'];

    // 1. CSRF Güvenlik Kontrolü (State Token Doğrulaması)
    if (receivedStateToken == null || receivedStateToken!=
```

```

originalStateToken) {
    _notifyUser(context, "Güvenlik doğrulaması başarısız. Yetkisiz
yönlendirme engellendi.", isError: true);
    return;
}

    if (transactionStatus == 'success' && receivedConsentId!= null) {
        // 2. QPAY ve BKM GEÇİT üzerinden rıza durumunun doğrulanması
        için arka plan servisi tetiklenir
        final bool isApiVerified = await
        _verifyConsentWithBackend(receivedConsentId);

        if (isApiVerified) {
            _notifyUser(context, "Banka hesap rızası başarıyla alındı.
Akaryakıt harcamalarınız eşitleniyor.");
            // Senkronizasyon motoru tetiklenerek card_transactions
            tablosu güncellenir.
        } else {
            _notifyUser(context, "Banka entegrasyonu onaylanamadı.",
            isError: true);
        }
        } else {
            _notifyUser(context, "Açık bankacılık izni kullanıcı veya banka
            tarafından i[span_10](start_span)[span_10](end_span)ptal edildi.",
            isError: true);
        }
    }

    Future<bool> _verifyConsentWithBackend(String consentId) async {
        try {
            final response = await http.post(

Uri.parse("https://api.depometrik.com/functions/v1/verify-consent"),
            headers: {"Content-Type": "application/json"},
            body: jsonEncode({"consent_id": consentId}),
        );
        if (response.statusCode == 200) {
            final Map<String, dynamic> data = jsonDecode(response.body);
            return data["verified"]?? false;
        }
        return false;
    } catch (_) {
        return false;
    }
}

    void _notifyUser(BuildContext context, String message, {bool isError
    = false}) {

```

```

ScaffoldMessenger.of(context).showSnackBar(
  SnackBar(
    content: Text(message),
    backgroundColor: isError? Colors.red : Colors.green,
    behavior: SnackBarBehavior.floating,
  ),
);
}
}

```

Sonuç ve Mimari Yol Haritası Değerlendirmesi

Faz 3 kapsamında tasarlanan bu hibrit entegrasyon modeli, DepoMetrik projesinin "çevrimdışı öncelikli" yapısını korurken, devlet servisleri ve bankacılık sistemleriyle güvenli ve gerçek zamanlı haberleşmesini mümkün kılmaktadır. Kimlik doğrulama süreçlerinde KVKK ve MASAK uyumluluğu, ham PII verilerinin saklanmaması ve yerel veritabanında SQLite3MultipleCiphers ile yüksek güvenli kriptolama teknikleriyle üst düzeyde sağlanmıştır.

Projenin bir sonraki aşaması olan Faz 4 (Donanım Entegrasyonu ve Kampanya Motoru), mobil cihazların Bluetooth BLE protokolü üzerinden ELM327 OBD-II dongle aygıtlarına bağlanmasını hedeflemektedir. Bu donanım entegrasyonu sayesinde elde edilecek gerçek zamanlı araç kilometre verileri, obd_readings tablosuna yazılacak ve manuel veri girişine gerek kalmadan tüketim hesaplama motorunun kilometre sayaç kısıtlamalarını dinamik olarak besleyecektir. Elde edilen finansal kart hareketleri ile anlık tüketim analizleri birleştirilerek, akıllı kampanya öneri algoritmaları devreye alınacaktır.

Alıntılanan çalışmalar

1. undefined -> v2.0.0 - (ÖHVPS) API İLKE VE KURALLARI, <https://ohvps.github.io/v2.0.0/contents/tanim-ve-kisitlemeler> 2. Ödeme Hizmetleri Veri Paylaşım Servisleri (ÖHVPS) - Bankalararası Kart Merkezi, <https://bkm.com.tr/urunler-ve-hizmetler/odeme-hizmetleri-veri-paylasim-servisleri/> 3. Encryption - Drift! - Simon Binder, <https://drift.simonbinder.eu/platforms/encryption/> 4. drift | Dart package - Pub.dev, <https://pub.dev/packages/drift> 5. Spring Boot SOAP client request response logger - GitHub, <https://github.com/kodgemisi/soap-logger-spring-boot-starter> 6. What is the solution to the error I encountered while using the soap module in Node.js, <https://stackoverflow.com/questions/74205450/what-is-the-solution-to-the-error-i-encountered-while-using-the-soap-module-in-n> 7. Web service via PL/SQL (ORA-29266: end-of-body reached error) - Oracle Forums, <https://forums.oracle.com/ords/apexds/post/web-service-via-pl-sql-ora-29266-end-of-body-reached-error-8692> 8. Make HTTP calls using the SOAP protocol - Postman Docs, <https://learning.postman.com/docs/use/send-requests/protocols/soap/making-soap-requests> 9. Searching the right format for SOAP access - Stack Overflow, <https://stackoverflow.com/questions/20786034/searching-the-right-format-for-soap-access> 10. HTTP requests (fetch) - Deno Docs, <https://docs.deno.com/deploy/classic/api/runtime-fetch/> 11. Fetch - Web documentation - Deno Docs, <https://docs.deno.com/api/web/~fetch> 12. @std/xml - Deno Docs, <https://docs.deno.com/runtime/reference/std/xml/> 13. Is supabase vault optimal for

saving personal data? - Reddit,
https://www.reddit.com/r/Supabase/comments/1taue71/is_supabase_vault_optimal_for_saving_personal_data/ 14. Security - Martech Stack Builder,
<https://www.martechstackbuilder.com/security/> 15. pgsodium (pending deprecation): Encryption Features | Supabase Docs, <https://supabase.com/docs/guides/database/extensions/pgsodium>
16. Column Encryption is SQL-only now · Changelog - Supabase,
<https://supabase.com/changelog/18849-column-encryption-is-sql-only-now> 17. Supabase Vault: Store Secrets Securely in Postgres - MakerKit, <https://makerkit.dev/blog/tutorials/supabase-vault>
18. Supabase Vault, <https://supabase.com/blog/supabase-vault> 19. Açık Bankacılık Basın Duyurusu - TCMB,
<https://tcmb.gov.tr/wps/wcm/connect/a5c14a85-374f-4a60-96c6-ce9eedc1f10c/DUY2022-48.pdf?MOD=AJPERES&CACHEID=ROOTWORKSPACE-a5c14a85-374f-4a60-96c6-ce9eedc1f10c-ojcBZ.E>